Java lambdas, streams and Optional

8 MAIN SECTIONS • PRINT-READY EDITION

In this guide

1. The three ideas in one minute 1
2. Lambdas and functional interfaces 2
3. Streams: how the pipeline runs 4
4. Optional: absent values without mystery 8
5. A complete payment example to trace 9
6. Interview questions with polished answers 10
7. Practice in 45 minutes 11
8. Final recall card 12

Interview companion for Sreedhar | Technical and coding round

Use this beside `interview-study-guide-full.md`. The examples use Java 8 syntax unless a newer release is named. Read one section, run or trace its example, then explain it aloud without looking. For a coding exercise, first write a clear loop; use a stream when it makes the transformation easier to read.

1. The three ideas in one minute

Lambda: A short way to supply behavior where Java expects a functional interface. Example: `n -> n > 0` tells `filter` which numbers to keep.

Stream: A pipeline that processes elements from a source. It can filter, transform, group or aggregate values. It does not store the data itself. Intermediate operations are lazy; a terminal operation asks for a result.

Optional: A container that says a result may be present or absent. It is especially useful as a return type for a lookup that may find nothing. It does not automatically eliminate all nulls.

30-second professional answer: "A lambda supplies a small piece of behavior through a functional interface. Streams let me express a sequence of transformations and a final result. Optional makes an absent return value explicit. I choose these tools when they make the business logic clearer, and I still pay attention to nulls, side effects, ordering and performance."

Simple picture: `source -> filter -> map -> collect`. The source supplies data; the middle stages describe what to do; the terminal stage produces the result. The lambda tells each stage how to handle one element.

2. Lambdas and functional interfaces

2.1 What a lambda really is

A lambda is an implementation of the single abstract method of a **functional interface**. Java infers the parameter and return types from the interface expected at that position. `@FunctionalInterface` asks the compiler to reject accidental extra abstract methods. Default and static methods do not prevent an interface from being functional.

JAVA

```java
@FunctionalInterface
interface PaymentRule {
    boolean accepts(Payment payment);
}

PaymentRule positive = p -> p.amount.signum() > 0;
boolean allowed = positive.accepts(payment);
```

The lambda has no independent type without a target. The expression `p -> ...` becomes a `PaymentRule` because that is the type on the left. A lambda can also be passed directly into a method that expects `Predicate<Payment>`.

Professional answer: "A lambda is an implementation of a functional interface's one abstract operation. Its target type tells the compiler what its inputs and result mean. I use it for small, local behavior; if the logic becomes long or has several responsibilities, I move it to a named method or class."

2.2 The four interfaces to know cold

Interface	Meaning	Example
`Predicate<T>`	`T -> boolean` : test a condition	`p -> p.successful`
`Function<T,R>`	`T -> R` : transform a value	`p -> p.accountId`
`Consumer<T>`	`T -> void` : perform an action	`p -> audit(p)`
`Supplier<T>`	`() -> T` : produce a value later	`() -> loadDefault()`

Also know `UnaryOperator<T>` (`T -> T`), `BinaryOperator<T>` (`(T,T) -> T`), `Comparator<T>` (ordering), and primitive variants such as `IntPredicate` and `ToIntFunction<T>` when boxing matters. `Predicate.and`, `or` and `negate` combine conditions. `Function.andThen` and `compose` combine transformations; their order differs.

JAVA

```
Predicate<Payment> success = p -> p.successful;
Predicate<Payment> usd = p -> "USD".equals(p.currency);
Predicate<Payment> successfulUsd = success.and(usd);
Function<String, String> trim = String::trim;
Function<String, String> upper = String::toUpperCase;
String code = trim.andThen(upper).apply(" usd "); // USD
```

Interview trap: A `Consumer` returns no value. `map` needs a `Function` that returns a transformed value. Do not use `forEach` just to build another list by mutating it; use `map(...).collect(...)` or a loop.

2.3 Lambda syntax and method references

JAVA

```
x ->x *2
        (a,b)->a.

compareTo(b)

p ->{

audit(p); return p.id; }
String::trim
Payment::getAmount
ArrayList::new
```

The method reference is a shorter spelling when an existing method already expresses the behavior. It does not make the code faster by definition. Use the form that makes input and output easiest to see. `Payment::getAmount` applied to a `Payment` works like `p -> p.getAmount()`.

Be precise: A lambda can capture a local variable only if the variable is final or effectively final, meaning you do not reassign it after initialization. This restriction applies to the local reference, not necessarily to the object it points to. Changing a captured mutable list is possible, but it can create confusing behavior or races.

JAVA

```
String tenant = "A";
Predicate<Payment> belongsToTenant =
        p -> tenant.equals(p.tenantId); // tenant is effectively final
```

Professional answer: "Captured local variables must be effectively final. That makes the value captured by the lambda stable. It does not make a captured mutable object thread-safe, so I avoid using shared mutable state inside parallel operations."

2.4 Comparators: a common coding question

JAVA

```
Comparator<Payment> byTimeThenId =
        Comparator.comparing((Payment p) -> p.createdAt)
                .thenComparing(p -> p.id);
```

Use `reversed()` when the entire comparison order should reverse. If only time should descend, reverse that component before adding the ID tie-breaker. Never compare integers by `a - b`; subtraction can overflow. Use `Integer.compare(a, b)` or comparator factories. A comparator must give consistent ordering, particularly when a sort or ordered map relies on it.

3. Streams: how the pipeline runs

3.1 Source, intermediate steps and terminal result

JAVA

```
List<String> accounts = payments.stream()       // source
        .filter(p -> p.successful)                    // intermediate
        .map(p -> p.accountId)                        // intermediate
        .distinct()                                   // intermediate
        .collect(Collectors.toList());               // terminal
```

`filter` keeps some values; `map` converts each kept `Payment` into an account ID; `distinct` removes equal IDs; `collect` creates the result. No processing needs to happen when you merely assemble the intermediate steps. The terminal operation starts traversal. Operations can process elements one by one rather than constructing a full list at every intermediate stage. Some steps, such as sorting, may need to buffer data.

Professional answer: "A stream pipeline consists of a source, intermediate operations and a terminal operation. Intermediate operations are lazy. This lets Java combine work and stop early for operations such as findFirst, but it also means I should not rely on side effects inside map or filter."

Important nuance: Java can optimize away invocation of some intermediate lambdas when their effects are irrelevant to the result. Use streams to compute values, not to guarantee an audit or external write. See Oracle's Stream contract.

3.2 Common operations in plain English

Operation	Plain English	Result or caution
`filter(predicate)`	Keep values that pass a test	Another stream
`map(function)`	Change each value into one new value	Another stream

Operation	Plain English	Result or caution
`flatMap(function)`	Change each value into a stream, then flatten	Another stream
`distinct()`	Keep one of each equal value	Uses equality rules
`sorted(comparator)`	Return values in an order	May buffer the input
`limit(n)`	Take at most `n` values	Often useful after ordering
`anyMatch`, `allMatch`, `noneMatch`	Ask a yes/no question	Short-circuiting terminal result
`findFirst()`	Return first matching value if present	Terminal `Optional<T>`
`count()`	Count values	Terminal `long`
`collect(...)`	Build a collection or grouped result	Terminal result
`reduce(...)`	Combine values into one value	Terminal result

Encounter order: A stream from a `List` normally has list order; a `HashSet` does not promise an order. `forEach` on a parallel stream does not promise encounter order; `forEachOrdered` preserves it at a potential cost. `findAny` may choose any element. Do not use an unordered source and promise a "first" record.

One-use rule: A stream is consumed by a terminal operation and should not be reused. Create a new stream from the source for another calculation. A collection can be reused; the stream is the traversal.

3.3 `map` versus `flatMap`: the interview favorite

Suppose each account has a list of payment IDs. `map` keeps one list per account; `flatMap` produces one combined stream of IDs.

JAVA

```
List<List<String>> nested = accounts.stream()
        .map(a -> a.paymentIds)
        .collect(Collectors.toList());

List<String> allIds = accounts.stream()
        .flatMap(a -> a.paymentIds.stream())
        .collect(Collectors.toList());
```

The first result looks like `[[P1, P2], [P3]]`. The second looks like `[P1, P2, P3]`. A `flatMap` mapper returns a **stream**; a `map` mapper returns one value.

Professional answer: "I use map for a one-to-one transformation and flatMap when each input produces zero or more outputs that I want in one stream. For example, map gives me a stream of payment lists from accounts; flatMap gives me one stream of payments."

3.4 `filter`, `findFirst` and early stopping

JAVA

```
Optional<Payment> firstFailure = payments.stream()
        .filter(p -> !p.successful)
        .findFirst();
```

On an ordered sequential stream, this asks for the first failed payment and need not inspect all later elements. `findFirst` returns an `Optional` because there may be no failure. If the input is unordered, "first" has no business meaning; define the ordering before relying on it. `limit` and matching operations can also stop early, though actual work and performance depend on the pipeline and whether it is parallel.

3.5 Collecting and grouping

JAVA

```
Map<String, List<Payment>> byAccount = payments.stream()
        .collect(Collectors.groupingBy(p -> p.accountId));

Map<String, Long> countByAccount = payments.stream()
        .collect(Collectors.groupingBy(
                p -> p.accountId, Collectors.counting()));
```

`groupingBy` collects values under a key. A downstream collector decides what each group contains. The first stores lists; the second stores counts. Group a financial amount by **account and currency**, or first confirm that conversion is intended. For a simple Java 8 total, a loop with `Map.merge` can be clearer than a nested collector.

`toMap` **trap:** If two elements produce the same key, `Collectors.toMap(keyMapper, valueMapper)` throws an exception. Provide a merge function only after deciding the business rule. For payment IDs, silently keeping the first duplicate may hide a serious problem.

JAVA

```
Map<String, String> latestByAccount = payments.stream()
        .collect(Collectors.toMap(
                p -> p.accountId,
                p -> p.id,
                (first, second) -> second));
```

The example keeps the **later encountered** ID, which is not necessarily the latest by timestamp. Sort with an explicit tie-breaker or compare timestamps in the merge function if the requirement is "latest." Know the difference before using this pattern in an interview.

3.6 Reduction and money

JAVA

```
BigDecimal total = payments.stream()
        .filter(p -> p.successful && "USD".equals(p.currency))
        .map(p -> p.amount)
        .reduce(BigDecimal.ZERO, BigDecimal::add);
```

The identity is the starting value and the result for an empty stream. Addition is suitable because combining groups in different orders gives the same mathematical total. In practice, validate scale, currency, reversals, duplicate records and the source of truth. Do not use `double` for a financial total. Keep a transaction total distinct from an account balance, which may require stronger consistency.

Professional answer: "Reduction combines elements into one result using an identity and an associative combining operation. For a monetary sum I use BigDecimal addition and only combine values with the same currency. I would verify the input's duplicate and reversal rules."

3.7 Primitive streams and boxing

`Stream<Integer>` holds boxed Integer references. `IntStream` specializes common numeric work and avoids some boxing overhead. A stream of objects can use `mapToInt`, `mapToLong` or `mapToDouble`. For money, do **not** switch to `double` merely to use a primitive stream; keep decimal arithmetic or an agreed minor-unit representation.

JAVA

```
int sum = numbers.stream().mapToInt(Integer::intValue).sum();
IntSummaryStatistics stats = numbers.stream()
        .mapToInt(Integer::intValue)
        .summaryStatistics();
```

3.8 Side effects, parallel streams and when to use a loop

This is risky, especially in parallel:

JAVA

```
List<String> ids = new ArrayList<>();
payments.

parallelStream().

forEach(p ->ids.

add(p.id));
```

`ArrayList` is not safe for concurrent writes, and `forEach` is not guaranteed to preserve order in a parallel stream. Prefer `map(...).collect(...)` for a result. Even a synchronized list would not turn an unclear side-effect pipeline into the best design. A parallel stream is not an automatic speed improvement: it adds coordination cost, may use a shared pool, and can behave poorly with blocking calls or small inputs. Measure a real workload before choosing it.

Use a **loop** when the operation has branching, checked exception handling, early `break` behavior, mutation of a complex result, or side effects whose order matters. Streams are a way to express a computation, not a requirement for modern Java.

Professional answer: "I avoid shared mutable state and external side effects in stream operations. I use sequential streams for clear data transformations and consider parallel execution only after checking data size, work per element, ordering requirements, and measured performance."

4. Optional: absent values without mystery

4.1 Creating and returning Optional

JAVA

```
Optional<String> present = Optional.of("A123");
Optional<String> missing = Optional.empty();
Optional<String> maybe = Optional.ofNullable(rawAccountId);
```

`of(null)` throws immediately. `ofNullable(null)` returns empty. An `Optional` reference itself should never be null. Optional is primarily useful for return values where absence is a normal outcome, such as `findAccount(id)`. Do not automatically put it in every field, parameter or persistence entity; follow the project's conventions and API requirements. See Oracle's Optional API.

Professional answer: "Optional makes a possibly absent result visible in a method's return type. I use it for legitimate absence, such as a lookup with no match. I do not use it to hide an error or to replace every nullable field."

4.2 `map` and `flatMap` on Optional

JAVA

```
Optional<String> region = findAccount(id)
        .map(account -> account.region);

Optional<String> managerEmail = findAccount(id)
        .flatMap(account -> findManager(account))
        .map(manager -> manager.email);
```

If `findManager` returns `Optional<Manager>`, `map` would produce `Optional<Optional<Manager>>`. `flatMap` avoids that extra layer. `Optional.map` treats a null mapper result as empty; `Optional.flatMap` expects the mapper to return a non-null Optional. Prefer not to return null from either mapper.

4.3 `orElse` versus `orElseGet` versus `orElseThrow`

JAVA

```
String a = maybeId.orElse("UNKNOWN");
String b = maybeId.orElseGet(() -> loadDefaultId());
String c = maybeId.orElseThrow(
        () -> new NoSuchElementException("Account ID missing"));
```

`orElse` takes a value. Java evaluates its argument before calling the method, even if the Optional is present. `orElseGet` takes a Supplier and runs it only when empty. For a cheap constant, `orElse` is simple. For an expensive lookup or an operation with side effects, `orElseGet` avoids unnecessary work. `orElseThrow` is for absence that is an error at that boundary. The no-argument `orElseThrow()` is newer than Java 8; the Supplier form above works in Java 8.

Interview trap: Calling `get()` without checking presence can throw. A pattern like `if (opt.isPresent()) return opt.get();` may be correct, but often `map`, `orElseGet` or `orElseThrow` expresses the rule more clearly. Do not use `orElse(null)` everywhere; that usually gives up the benefit of Optional at the next line.

4.4 Optional and error semantics

"No account found" can be a normal lookup result (`Optional.empty()`), a 404 at an HTTP boundary, or an invalid state in a payment operation. Choose based on context. A database outage is **not** an empty result. Swallowing an exception and returning `Optional.empty()` would make an operational failure look like missing data.

Professional answer: "I reserve an empty Optional for expected absence. I propagate or translate real failures separately so callers can distinguish 'not found' from 'could not check.'"

5. A complete payment example to trace

The following snippet assumes a `Payment` with `accountId`, `currency`, `amount` and `successful` fields. It totals successful USD payments by account. `Map.merge` is an intentional loop alternative because it makes the accumulation and duplicate policy easy to inspect.

JAVA

```
Map<String, BigDecimal> totals = new HashMap<>();
for(
Payment p :payments){
        if(!p.successful ||!"USD".

equals(p.currency))continue;
        totals.

merge(p.accountId, p.amount, BigDecimal::add);
}
```

A stream can express the same grouping through a downstream reduction:

JAVA

```
Map<String, BigDecimal> totals = payments.stream()
        .filter(p -> p.successful && "USD".equals(p.currency))
        .collect(Collectors.groupingBy(
                p -> p.accountId,
                Collectors.reducing(
                        BigDecimal.ZERO, p -> p.amount, BigDecimal::add)));
```

Both versions assume the input has the right tenant scope and already handles duplicate events according to business rules. Both can be wrong for a payment system if those assumptions are false. The loop may be preferable in a live coding interview because it is short and easy to test. Tell the interviewer why you chose it.

Question to rehearse: "How would you adapt this for multiple currencies?" Answer: use a composite key containing account and currency, or a nested map. Define refunds and reversals. Do not add USD and EUR amounts directly.

6. Interview questions with polished answers

What is a functional interface?

"It has one abstract method, so a lambda can supply that method's implementation. Predicate, Function, Consumer and Supplier are common examples. The annotation is optional, but useful because the compiler checks my intent."

What is lazy evaluation in a stream?

"Intermediate operations describe work but do not produce a result by themselves. A terminal operation starts traversal. This can avoid unnecessary work; for example, findFirst can stop once it finds a match in an ordered sequential stream."

What is the difference between map and flatMap?

"Map returns one result for each input. FlatMap handles an input that produces a stream of results and combines those streams into one. For account-to-payments, map gives me lists of payments; flatMap gives me one sequence of payments."

How is Optional different from null?

"Optional makes possible absence part of the return type and provides methods to handle it. It does not guarantee that every value in the system is non-null. I use it for expected absence and keep actual failures separate."

When would you avoid streams?

"I use a loop when it makes state changes, early exit, exception handling or branching clearer. I also avoid side effects inside a stream pipeline. Readability and correctness matter more than making every operation a stream."

Are streams faster than loops?

"Not inherently. A stream can express a transformation clearly and may optimize traversal, but it also has overhead. Parallel streams add coordination and ordering costs. I choose based on clarity first and measure performance for important paths."

Why can `toMap` fail?

"The two-argument toMap collector throws if two inputs generate the same key. I decide whether duplicates are invalid, should be grouped, or need an explicit merge rule. In financial data, I would not silently discard one record."

Why is a parallel stream with a shared ArrayList unsafe?

"Multiple threads can modify the list at the same time, causing lost or corrupted updates. I would collect the result using a suitable collector or stay sequential. I would also check whether parallel execution actually helps."

7. Practice in 45 minutes

First 15 minutes - trace: For `[" a ", "", " b ", " a "]`, predict the result of trim, remove empty, uppercase, distinct and collect in that order. Explain what changes if you put `distinct` before trim.

Next 15 minutes - code: Given payments, produce the IDs of successful USD payments in encounter order. Then group successful payments by account and currency and total them with BigDecimal. Write a loop first, then a stream version. Add empty input, duplicates and a reversal to your test cases.

Final 15 minutes - speak: Answer the eight questions above aloud. Keep each answer to 30-45 seconds. Repeat only the answers you could not explain clearly.

Quick self-check

1. What type does `p -> p.amount` need for `map`? **Function from Payment to BigDecimal.**
2. What does `flatMap(a -> a.payments.stream())` return? **One stream of payments, rather than a stream of lists or streams.**
3. Does `.filter(...).map(...)` run without a terminal operation? **No traversal is requested yet.**
4. Is `Optional.of(null)` empty? **No; it throws. Use `ofNullable` when null is possible.**
5. Will `present.orElse(expensive())` call `expensive()`? **Yes. Java evaluates the argument first.**
6. Can `toMap` accept duplicate keys automatically? **No; choose a merge rule or grouping collector.**

7. Does `parallelStream()` guarantee faster code or ordered `forEach`? **No.**
8. Is a database error an empty Optional? **No; keep failures distinct from absence.**

8. Final recall card

Lambda = behavior passed through an interface. Predicate tests, Function transforms, Consumer acts, Supplier produces later. Captured locals are effectively final.

Stream = source + lazy intermediate steps + terminal result. `filter` keeps; `map` transforms one-to-one; `flatMap` flattens one-to-many. `collect` builds; `reduce` combines. A stream is generally used once. Avoid stateful side effects.

Optional = present or absent return value. `of` requires non-null; `ofNullable` accepts null; `map` transforms; `flatMap` avoids nested Optionals; `orElseGet` computes a fallback only when needed. An error is not the same as absence.

Sources checked: Oracle Stream API, Optional API, functional interface and lambda tutorial, and stream package guide. The tutorial was written for JDK 8; check your project's release for newer convenience methods.